

Projet Informatique de 2ème année

Groupe 32

EX-Libris

Étudiants :

Arnaud BICHON
Axel LECLERCQ
Cécile LOCATELLI
Moussa SOULAMA
Anne-Camille TAMPÉ
Paul FOURCADE

Encadrant :

Elwenn JOUBREL

Table des matières

1	Introduction	3
2	Cahier des Charges	4
2.1	Présentation d'Open Library	4
2.2	Diagramme de cas d'utilisation	6
2.3	Diagrammes d'activités	9
3	Organisation du projet	11
3.1	Organisation du groupe	11
3.2	Diagramme de Gantt	12
4	Maquette de l'application	13
4.1	Modèle des données	13
4.2	Diagramme de classes	15
4.2.1	Classes Business Object	15
4.2.2	Diagramme de classe pour l'authentification	17
4.2.3	Diagramme de classe pour les critiques	19
4.3	Diagrammes de séquence	21

1 Introduction

Depuis plusieurs années, le numérique a profondément transformé la manière dont nous consommons et partageons nos loisirs culturels. Le cinéma dispose de Letterboxd, la musique de Last.fm : des plateformes communautaires qui permettent à chacun de garder une trace de ses découvertes, de les noter et d'échanger avec d'autres passionnés. La lecture, pourtant, reste étonnamment mal servie sur ce plan. Le principal service existant, Goodreads, appartient à Amazon, ce qui pose des questions en matière d'indépendance éditoriale et d'exploitation des données personnelles des lecteurs.

C'est de ce constat qu'est né le projet Ex-Libris : une application de suivi de lecture pensée comme un véritable réseau social pour les lecteurs. L'objectif n'est pas de reconstruire un catalogue de livres, un travail déjà accompli et maintenu par la plateforme collaborative Open Library, qui référence plusieurs millions d'ouvrages en accès libre. Ce qu'Open Library ne propose pas, en revanche, c'est la dimension humaine de la lecture : qui a lu quel livre, quand, avec quel avis, et quelles recommandations peuvent en être tirées. C'est précisément cette couche que notre application vient ajouter.

Concrètement, Ex-Libris doit permettre à un utilisateur de créer un compte, de rechercher un livre et d'en consulter la fiche détaillée, de l'ajouter à sa bibliothèque personnelle avec un statut de lecture, de le noter et d'en rédiger une critique. Au-delà de cette gestion individuelle, l'application intègre une dimension sociale : chaque utilisateur peut consulter le profil d'un autre lecteur, s'abonner à son activité, et bénéficier d'un système de recommandation construit à partir des affinités entre utilisateurs.

D'un point de vue technique, le projet repose sur une API REST développée avec FastAPI, exploitant les données de l'API publique Open Library et s'appuyant sur une base de données PostgreSQL pour la persistance des données propres à notre application. Le langage retenu est Python, dans une logique de programmation orientée objet.

Ce dossier présente dans un premier temps le cahier des charges du projet, les choix et arbitrages qui ont guidé nos réflexions, ainsi que l'organisation retenue au sein de notre équipe. Il détaille ensuite le modèle de conception élaboré pour répondre à ces besoins, à travers les différents diagrammes UML réalisés.

2 Cahier des Charges

Ce cahier des charges exprime le besoin initial auquel notre application doit répondre : une application de suivi de lecture combinant la richesse du catalogue d'Open Library à une véritable dimension sociale entre lecteurs. Sur cette base, notre travail a consisté à reformuler ce besoin, à en préciser le périmètre exact, et à définir les fonctionnalités attendues ainsi que les contraintes techniques qui en découlent, notamment celles liées à l'exploitation d'une source de données externe.

2.1 Présentation d'Open Library



Notre application repose entièrement sur les données fournies par Open Library, une base collaborative et gratuite qui référence plusieurs millions d'ouvrages. Il s'agit d'une API web : on l'interroge en construisant des adresses (URLs) spécifiques, et elle nous renvoie les données demandées au format JSON.

Besoin	URL type	Exemple
Rechercher des livres	<code>search.json?q=...</code>	<code>openlibrary.org/search.json?q=le+petit+prince</code>
Voir une œuvre précise	<code>/works/{id}.json</code>	<code>openlibrary.org/works/OL10263W.json</code>
Voir une édition précise	<code>/books/{id}.json</code>	<code>openlibrary.org/books/OL7366018M.json</code>
Voir un auteur précis	<code>/authors/{id}.json</code>	<code>openlibrary.org/authors/OL31901A.json</code>
Voir la définition officielle d'un type	<code>/type/{nom}</code>	<code>openlibrary.org/type/work</code>

Pour bien comprendre comment exploiter ces données, il est essentiel de saisir la façon dont Open Library organise ses informations. La base distingue en réalité trois notions différentes qui, dans le langage courant, sont toutes désignées par le mot “livre” :

- L'auteur (Author) : la personne ayant écrit l'œuvre.
- L'œuvre (Work) : il s'agit de l'idée générale du livre, indépendamment de toute édition particulière. C'est à ce niveau que l'on trouve par exemple le titre, un résumé, ou les grands thèmes abordés.
- L'édition (Edition) : il s'agit d'une publication précise et matérielle de cette œuvre, par exemple une édition Harcourt de 1971. Une même œuvre peut avoir des centaines d'éditions différentes (traductions, rééditions, formats poche...).

Dans notre base de données, nous avons fait le choix de conserver, pour chaque œuvre, l'ensemble de ses éditions, afin d'offrir aux utilisateurs une réelle variété de choix (langue, édition, format) lorsqu'ils ajoutent un livre à leur bibliothèque. Nous détaillerons la façon

dont nous avons stocké et traité ces données dans la suite de ce dossier d'analyse, lors de la conception de la maquette.

Nous avons également constaté que certaines données présentes dans Open Library manquent de cohérence, notamment les thèmes et genres associés aux œuvres (champ subjects), qui sont rédigés en texte libre et mélangent parfois de véritables genres littéraires avec de simples mots-clés. Cette hétérogénéité a directement motivé l'une de nos réflexions pour la fonctionnalité de normalisation des genres, qui vise à proposer une classification plus cohérente et plus facilement exploitable pour la recherche par genre.

Type Work

Champ	Description
title	Titre de l'œuvre
subtitle	Sous-titre
authors[]	Auteur(s) + leur rôle (via author_role)
subjects[]	Thèmes/genres (peu normalisés)
subject_places[]	Lieux liés à l'œuvre
subject_times[]	Époques liées à l'œuvre
subject_people[]	Personnes liées à l'œuvre
description	Résumé/description
dewey_number[]	Classification Dewey (bibliothéconomie)
first_sentence	Première phrase de l'œuvre
original_languages[]	Langue(s) d'origine
first_publish_date	Date de toute première publication
cover_edition	Édition choisie pour la couverture affichée
covers[]	Identifiants d'images de couverture
genres[]	Tags de classification (système différent de subjects)
identifiers	Liens vers bases externes (Wikidata, Goodreads...)

Type Edition

Champ	Description
title	Titre de cette édition
authors[]	Auteur(s) (référence directe, pas de rôle ici)
isbn_10 / isbn_13	Identifiant international, propre à cette édition
number_of_pages	Nombre de pages de cette édition précise
publishers[]	Éditeur(s)
publish_date	Date de CETTE publication
physical_format	Format (poche, relié, cartonné...)
languages[]	Langue de cette édition/traduction
works[]	Lien retour vers le Work parent
subjects[] / genres[]	Existent officiellement mais rarement remplis à ce niveau
series[]	Appartenance à une saga/collection
translation_of	Indique si c'est une traduction
source_records[]	Traçabilité des sources ayant alimenté la fiche

Type Author

Champ	Description
name / personal_name	Nom de l'auteur
title	Attention : ici ce n'est PAS un titre de livre, mais le nom sous forme "Nom, Prénom" (tri alphabétique)
alternate_names[]	Variante/orthographe du nom rencontrées dans les sources
bio	Biographie
birth_date / death_date	Dates de naissance/mort
photos[]	Identifiants de photos
remote_ids	Liens vers bases externes (VIAF, Wikidata, ISNI)
location	Lieu associé à l'auteur
wikipedia	Lien Wikipedia

2.2 Diagramme de cas d'utilisation

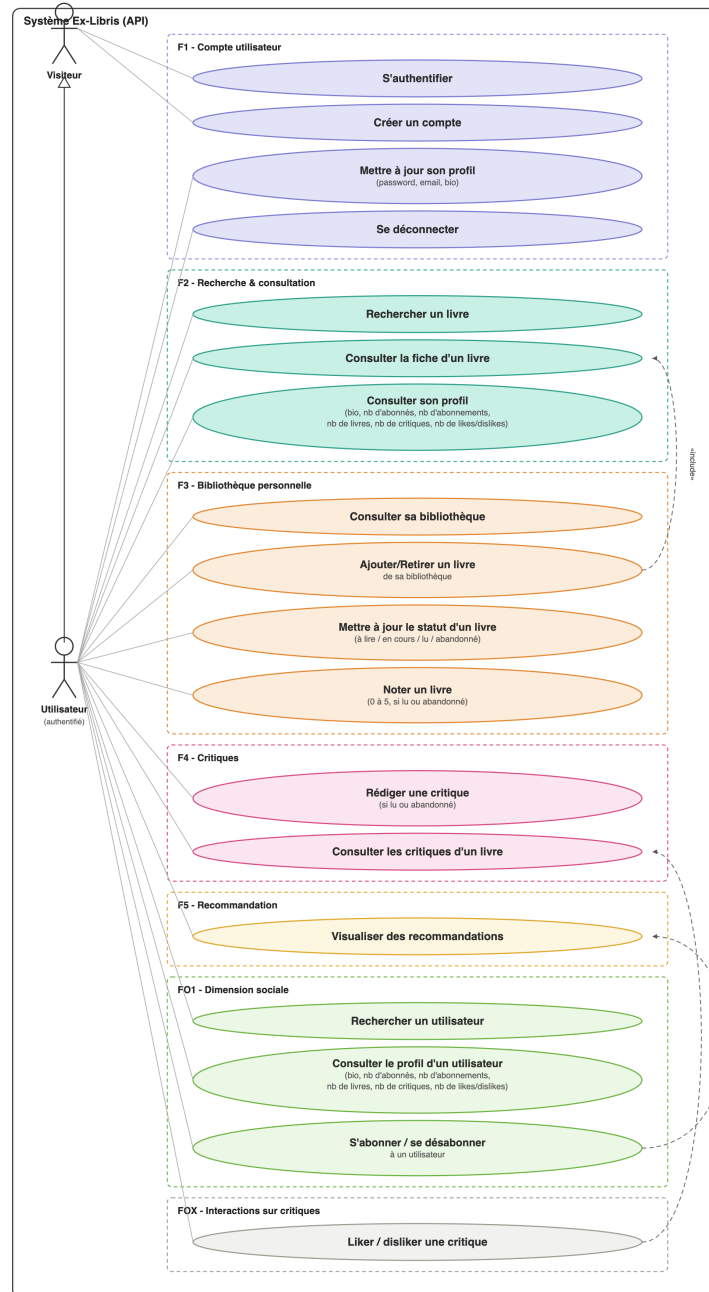


Figure 11 – Diagramme de cas d'utilisation

Le système Ex-Libris distingue deux acteurs : le **Visiteur**, non authentifié, et l'**Utilisateur**,

qui hérite du Visiteur et accède à l'ensemble des fonctionnalités une fois connecté. Le Visiteur ne peut que créer un compte ou s'authentifier ; toutes les autres actions nécessitent d'être connecté.

Partant de nos expériences personnelles d'utilisateurs de sites de musique ou de films, nous avons réfléchi ensemble aux fonctionnalités que nous souhaitions voir figurer dans notre projet.

Les cas d'utilisation sont regroupés en sept paquetages fonctionnels. F1 à F5 couvrent le cœur de l'application — gestion du compte, recherche et consultation des livres, bibliothèque personnelle, critiques et recommandations —, tandis que FO1 et FOX portent les fonctionnalités optionnelles à dimension sociale (abonnements) et d'interaction sur les critiques (like/dislike), envisagées comme des extensions si le temps le permet.

Plusieurs choix de modélisation méritent d'être précisés :

- **Consulter un profil / Consulter son profil.** Ces deux cas d'utilisation apparaissent dans des paquetages différents afin de distinguer la consultation de son propre profil de celle du profil d'un autre utilisateur. Dans les deux cas, le périmètre est volontairement allégé pour éviter de multiplier les cas d'utilisation : l'affichage proposera le pseudo, la bio de l'utilisateur, ainsi que le nombre de ses livres, de ses critiques, de ses abonnements et de ses abonnés. Depuis cet écran, l'utilisateur pourra consulter la bibliothèque, la liste des critiques, la liste des abonnés ou celle des abonnements. Le détail des parcours de navigation entre ces écrans relève du diagramme d'activité. Notons que, lorsque l'utilisateur consulte son propre profil, il a en outre la possibilité d'en modifier certains éléments, comme sa bio.
- **Note et critique.** Il s'agit de deux cas d'utilisation distincts (F3 « Noter un livre » vs F4 « Rédiger une critique »), ce qui reflète notre modèle de données : la note est un attribut de Lecture, indépendant de l'objet Critique. Un utilisateur peut donc noter un livre sans rédiger de critique, et inversement. Dans les deux cas, nous considérons qu'un utilisateur ne peut noter ou rédiger une critique que pour un livre de sa bibliothèque portant le statut « lu » ou « abandonné ».
- **Like/dislike.** Lorsqu'un utilisateur consulte la critique d'un livre, il peut la liker ou la disliker, que ce livre appartienne ou non à sa propre bibliothèque.
- **Visualiser des recommandations.** À ce stade, la logique de recommandation n'est pas totalement arrêtée. Le principe retenu est d'afficher plusieurs sections de recommandations générées automatiquement (par exemple : « de vos auteurs favoris », « les populaires d'Ex-Libris »), combinant différents critères — dont le réseau d'abonnements de l'utilisateur (F5B), ce qui justifie la relation «extend» entre « S'abonner/se désabonner à un utilisateur » et « Visualiser des recommandations ». Le détail de ces critères et leur articulation seront précisés dans le diagramme de séquence présenté plus loin dans ce rapport.

Deux relations complètent le diagramme : une relation «include» entre « Ajouter/retirer un livre de sa bibliothèque » et « Consulter la fiche d'un livre » (on ne peut ajouter un livre qu'après avoir consulté sa fiche), et de même entre « Liker/disliker une critique » et « Consulter les critiques d'un livre ». Une relation «extend» relie « S'abonner/se désabonner à un utilisateur » à « Visualiser des recommandations », le réseau d'abonnements constituant l'un des critères entrant dans la composition de cette page (cf. ci-dessus).

2.3 Diagrammes d'activités

Diagramme d'activité général

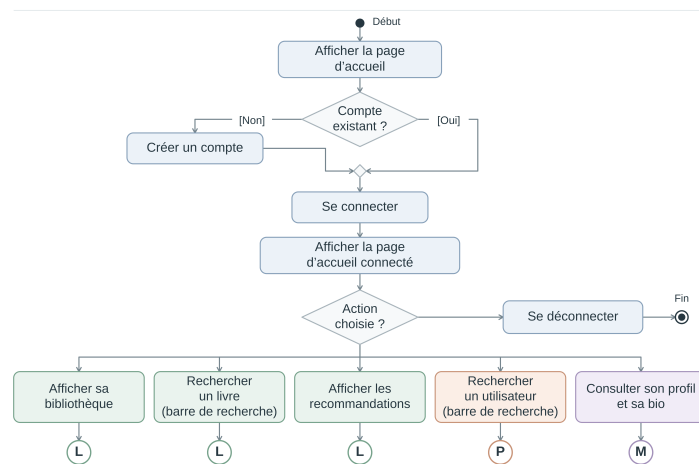


Figure 11 – Diagramme d'activité : parcours utilisateur global

Consulter mon profil

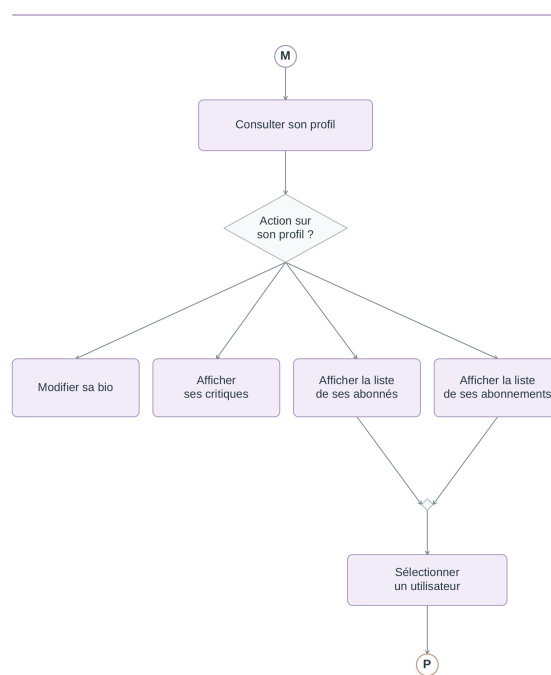


Figure 11 – Diagramme d'activité : Diagramme d'activité : gestion du profil personnel

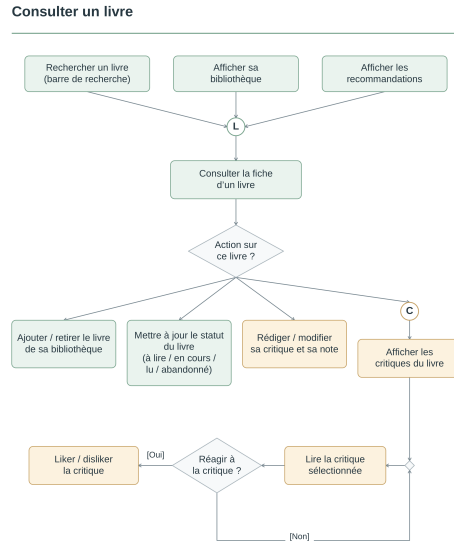


Figure 11 – Diagramme d'activité : consultation d'un livre

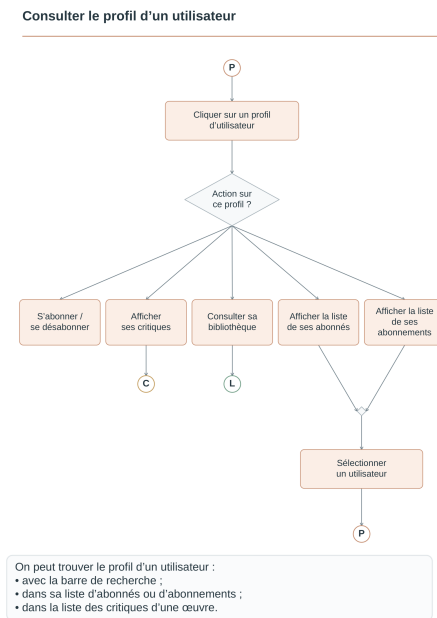


Figure 11 – Diagramme d'activité : inscription et authentification

3 Organisation du projet

3.1 Organisation du groupe

Dès le début du projet, nous avons désigné Anne-Camille Tampet comme cheffe de projet et Cécile Locatelli comme leader technique, en veillant, conformément aux conseils donnés en début de projet à ne pas cumuler ces deux rôles sur une même personne.

Pour communiquer, nous avons créé un groupe sur Teams, sur lequel chacun peut partager ses avancées, poser une question ou signaler un point de blocage dès que le besoin s'en fait sentir. C'est notre principal canal d'échange, sur lequel nous discutons régulièrement. En parallèle, nous avons mis en place un dépôt partagé sur GitHub, accessible à tous les membres du groupe, qui nous sert à centraliser et à distribuer l'ensemble des fichiers importants du projet.

Concernant la répartition du travail, notre organisation a surtout été pensée pour cette première phase d'analyse. Chaque membre du groupe s'est vu confier la réalisation d'un diagramme ou d'une partie spécifique du dossier, de façon à ce que chacun puisse s'investir concrètement dans la réflexion et la conception du projet. Cette répartition individuelle ne nous a cependant pas empêchés de travailler de façon très collective. Nous avons pris l'habitude de nous retrouver au minimum deux fois par semaine pour faire le point ensemble. Ces moments nous permettaient de relire et de discuter les diagrammes réalisés par chacun, de nous assurer que tout le monde comprenait bien les choix faits par les autres et de les faire évoluer si nécessaire. Cette phase de discussion collective a été particulièrement importante en tout début de projet. Elle nous a permis de poser une base commune solide et de nous assurer que tout le monde partageait la même vision de l'application avant d'avancer plus en détail sur la partie codage.

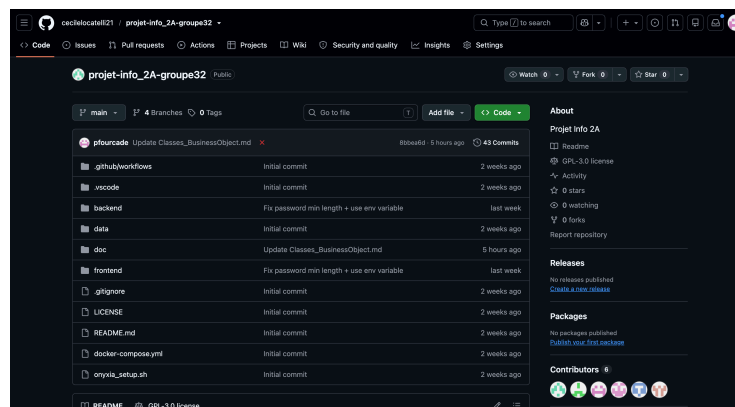


Figure 11 – Dépôt Github

3.2 Diagramme de Gantt

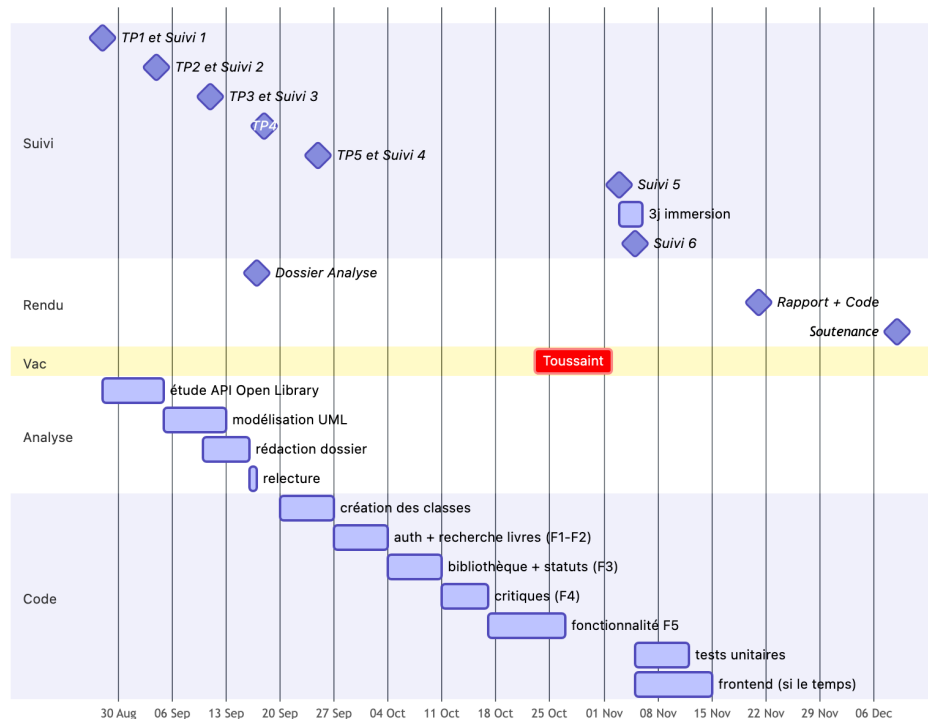


Figure 11 – Diagramme de Gantt

Notre planning se découpe en deux grandes phases. D'abord la phase d'analyse, de fin août à fin septembre : nous commençons par étudier l'API Open Library, puis nous réalisons l'ensemble de la modélisation UML afin de bien comprendre comment notre application va s'articuler et comment la construire.

Une fois cette base posée, nous passons à la phase de code, d'octobre à début novembre. Nous avons choisi de développer les fonctionnalités par ordre de difficulté croissante : d'abord les classes de base, puis l'authentification et la recherche de livres (F1-F2), la bibliothèque avec les statuts de lecture (F3), les critiques (F4), et enfin notre fonctionnalité F5 (les recommandations), qui nécessitera davantage de temps du fait de sa complexité. Les tests unitaires et le frontend interviendront en dernier, le frontend n'étant pas prioritaire, sera développé uniquement s'il nous reste du temps.

4 Maquette de l'application

Après avoir défini les besoins et les fonctionnalités attendues, place à la conception technique de notre application. Nous détaillons ici l'organisation de la base de données, la structure en couche des classes de l'application (controller, service, dao), ainsi que le fonctionnement de quelques échanges clés entre les différents composants du système.

4.1 Modèle des données

La conception de la base de données repose sur la séparation entre les informations bibliographiques des livres et les données propres aux utilisateurs de l'application. Les informations générales concernant les livres sont récupérées à partir de l'API Open Library : il n'est donc pas nécessaire de reconstruire localement un catalogue complet. La base de données est principalement destinée à conserver les utilisateurs ainsi que leurs interactions avec les livres (lectures, critiques, appréciations) et les relations entre utilisateurs à travers le système d'abonnement.

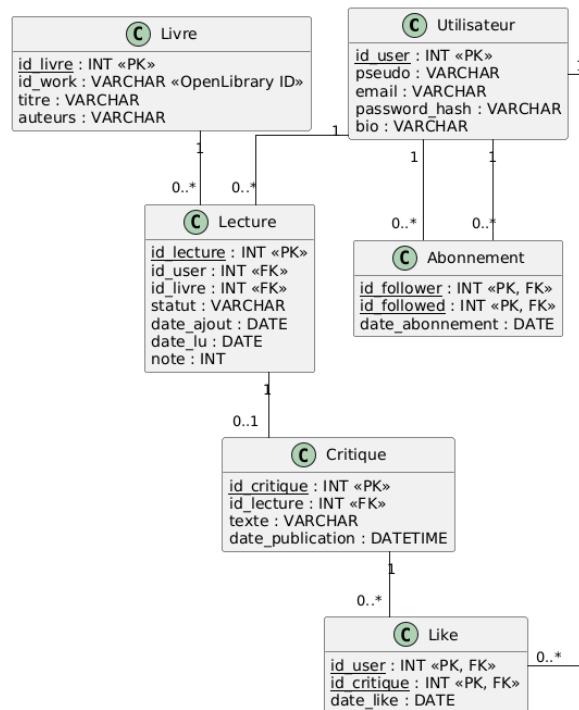


Figure 11 – Modèle des données

La base est composée de six tables : Utilisateur, Livre, Lecture, Critique, Abonnement et Like.

Table Utilisateur

La table Utilisateur représente les personnes utilisant l'application. Elle contient un identifiant unique `id_user` (SERIAL), clé primaire, ainsi que `pseudo`, `email`, `password_hash` et `bio`. Cette table constitue une entité centrale de la base puisque les fonctionnalités personnalisées (gestion des lectures, publication de critiques, abonnements) sont toutes associées à un utilisateur.

Table Livre

La table Livre représente les ouvrages enregistrés dans l'application. Elle possède un identifiant unique `id_livre` (SERIAL), clé primaire. L'attribut `id_work` (VARCHAR) correspond à l'identifiant de l'œuvre fourni par Open Library et permet d'établir le lien avec les informations bibliographiques de l'API.

Bien que `id_work` permette déjà d'identifier une œuvre dans Open Library, nous avons choisi de définir notre propre clé primaire `id_livre`. Cela permet à notre base de disposer d'un identifiant interne indépendant de l'API externe : `id_work` sert alors de référence vers Open Library, tandis que `id_livre` constitue la référence interne utilisée par les autres tables, notamment Lecture.

Les attributs `titre` et `auteurs` permettent de conserver localement les principales informations bibliographiques, afin de les consulter rapidement sans requête systématique à Open Library, et de limiter le nombre d'appels à l'API (qui impose une limitation du nombre de requêtes par seconde). Les informations bibliographiques supplémentaires, non nécessaires au fonctionnement immédiat, restent récupérées directement auprès d'Open Library lorsque cela est nécessaire.

Table Lecture

La table Lecture représente les interactions de lecture entre les utilisateurs et les livres. Elle possède un identifiant unique `id_lecture` (SERIAL), clé primaire, et deux clés étrangères : `id_user` (référence Utilisateur) et `id_livre` (référence Livre). Elle contient également `statut` (état de lecture : pas commencé, en cours, lu, abandonné), `date_ajout`, `date_lu` et `note`.

Le recours à cette table permet de représenter une relation plusieurs-à-plusieurs entre les utilisateurs et les livres, tout en conservant les informations propres à chaque interaction.

Table Critique

La table Critique permet de gérer les avis publiés sur les livres. Elle possède un identifiant unique `id_critique` (clé primaire) et une clé étrangère `id_lecture`, qui rattache chaque critique à une lecture enregistrée plutôt qu'à un livre ou un utilisateur directement. Les autres attributs sont `texte` et `date_publication`.

D'après le diagramme, une lecture peut recevoir au maximum une critique (0..1), et une critique est obligatoirement associée à une seule lecture. L'identifiant propre à la critique est également nécessaire pour la gestion des appréciations, une critique étant un objet identifié sur lequel les utilisateurs peuvent interagir.

Table Like

La table Like représente les appréciations associées aux critiques. Elle contient deux clés étrangères, `id_user` et `id_critique`, qui forment ensemble la clé primaire composite (évitant qu'un même utilisateur enregistre plusieurs fois la même appréciation pour une critique donnée). Elle contient également `date_like` et `aime` (BOOL, TRUE pour « j'aime », FALSE pour « je n'aime pas »).

Cette table matérialise une relation plusieurs-à-plusieurs entre les utilisateurs et les critiques.

Table Abonnement

La table Abonnement représente les relations entre utilisateurs, en mettant en relation deux instances de la même entité Utilisateur selon deux rôles différents : `id_follower` (l'utilisateur qui s'abonne) et `id_followed` (l'utilisateur suivi), qui forment ensemble la clé primaire composite. La date de l'abonnement est conservée dans `date_abonnement`.

4.2 Diagramme de classes

4.2.1 Classes Business Object

Certaines classes contiennent les attributs des objets qui sont utilisés pour les méthodes présentes dans les classes DAO et Services. Ces classes définissent les business object qui sont les objets manipulés pour construire l'application.

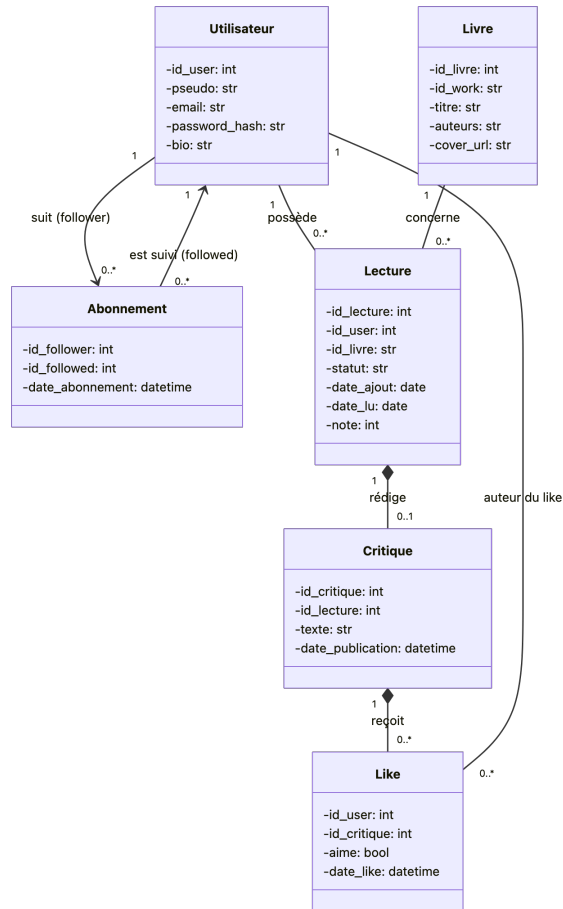


Figure 11 – Diagramme de classes : Business Object

Chaque classe de ce diagramme correspond à une entité identifiée dans notre modèle de données (BDD) : **Utilisateur**, **Livre**, **Lecture**, **Critique**, **Abonnement**, **Like**. Ce sont des classes qui ne contiennent que des attributs, aucune méthode métier. Ce choix découle directement de l'architecture en couches imposée par le projet (**controller** → **service** → **dao**) : un business object représente une donnée et non un traitement. La logique (vérifications, calculs, orchestration) est déléguée aux services, ce qui évite de mélanger « ce que sont les données » et « ce qu'on en fait ».

La classe **Livre** stocke les attributs récupérés sur la plateforme OpenLibrary et qui sont nécessaires en même temps et ne peuvent pas être disponibles dans un temps limité à cause de la limite de trois requêtes par seconde qui est donnée par OpenLibrary. On envisage également de récupérer aussi l'url de la couverture du livre qui sera stockée en tant qu'url ou d'image en fonction de l'accessibilité du fichier, dans la mesure du possible.

La classe **Lecture** permet de mettre un livre dans sa bibliothèque, de lui ajouter un statut (à lire, en cours, lu, abandonné). **date_ajout** correspondra à la date de création de l'objet **Lecture**. La date **date_lu** sera la date à laquelle le statut est passé à "lu".

La classe **Utilisateur** contient les attributs liés à l'authentification des utilisateurs : l'application donne à chaque utilisateur un id unique. L'utilisateur renseigne son pseudo, son email (respect du format xxx@xx.xx), sa bio et son mot de passe en format hashable.

Un utilisateur peut avoir plusieurs **Lecture**, mais chaque **Lecture** a un seul utilisateur.

Les utilisateurs peuvent s'abonner les uns aux autres. Cette information est renseignée dans la classe **Abonnement** qui à chaque **id_abonnement** associe un utilisateur qui s'abonne (follower) et un utilisateur à qui on s'abonne (followed). Nous avons besoin de l'**id_abonnement** car un abonnement n'est pas forcément réciproque. L'abonnement permet d'avoir sur son fil d'actualité les nouvelles lectures des utilisateurs auxquels on s'est abonné.

La classe **Critique** recense les critiques proposées par les utilisateurs pour une lecture (un livre qu'ils ont lu ou abandonné). La date de publication doit être postérieur à la date de début de lecture de la classe "Lecture". Un utilisateur peut rédiger plusieurs critiques et un livre peut recevoir plusieurs critiques. Une critique ne porte que sur une lecture.

Quand un utilisateur (**id_user**) consulte une critique, il peut mettre un ' j'aime ' ou ' je n'aime pas ' à cette critique (**id_critique**). Ces informations sont des attributs de la classe **Like**.

4.2.2 Diagramme de classe pour l'authentification

Pour s'authentifier, l'utilisateur va renseigner des informations qui correspondent à l'objet de la classe **Utilisateur**.

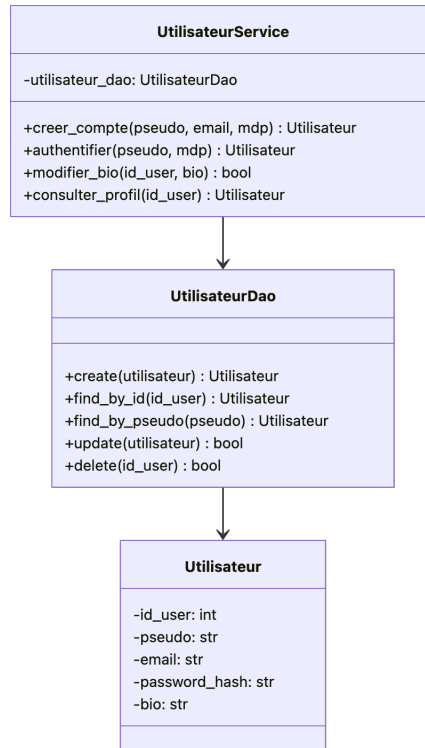


Figure 11 – Diagramme de classes : authentification

La classe “UtilisateurDAO” permet de créer, de trouver, de modifier ou de supprimer un objet de la classe “Utilisateur”. Cette classe est en lien avec les données stockées en local dans la BDD car ces informations ne relèvent pas de OpenLibrary. La méthode update permet de modifier toutes les informations d’un utilisateur.

La classe “UtilisateurService” répond aux besoins métiers de l’application : les méthodes créer un compte, s’authentifier et consulter un profil renvoient l’objet Utilisateur et la méthode modifier_bio renvoie un booléen (False si l’utilisateur n’existe pas ou si la sauvegarde en base échoue).

4.2.3 Diagramme de classe pour les critiques

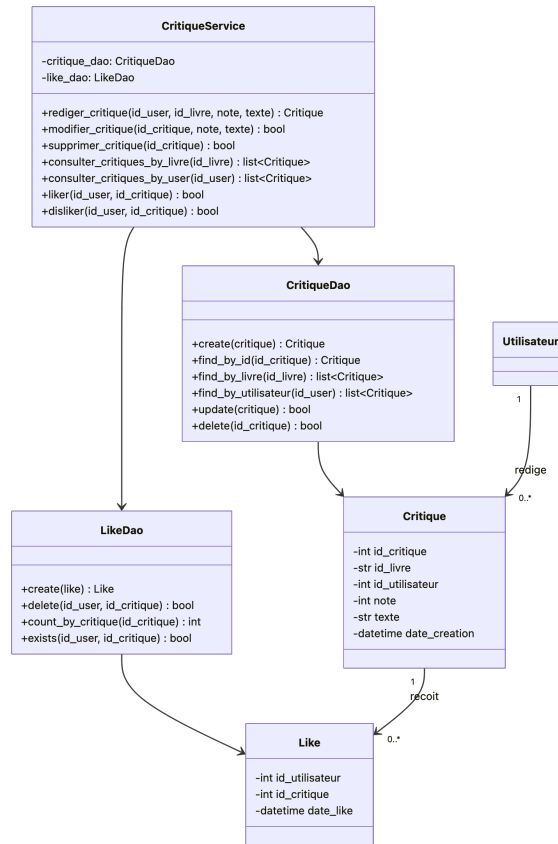


Figure 11 – Diagramme de classes : critique

La classe “CritiqueDao” permet de faire le lien entre les objets de la classe “Critique” en mémoire et la table critique dans la base de données. La méthode ‘create’ permet de créer une nouvelle critique dans la base, ‘find_by_id’ permet de récupérer une critique précise, ‘find_by_livre’ permet de récupérer toutes les critiques d’un livre (toutes éditions confondues), ‘update’ réécrit une critique existante (possibilité seulement pour l’utilisateur qui l’a écrite), et ‘delete’ supprime la critique et tous ses likes avec.

La classe “LikeDao” est l’équivalent de “CritiqueDao” mais pour la table like de la base de données. ‘create’ insère un nouveau like dans la base de données. ‘delete’ supprime un like précis (méthode appelée aussi quand un utilisateur passe du like au dislike ou vice versa). ‘count_by_critique’ compte le nombre de like par critique. ‘exists’ vérifie si un utilisateur a déjà liké ou disliké une critique.

La classe “CritiqueService” combine les données alors que “CritiqueDao” exécute seulement des requêtes. Elle s’appuie sur “CritiqueDao” et “LikeDao” pour proposer les fonctionnalités

attendues dans l'application. ' Rediger_critique ' fait appel à CritiqueDao.create en vérifiant les contraintes métier (note entre 1 et 5). ' Modifier_critique ' et ' supprimer_critique ' vérifient aussi que l'action soit exécutée par l'utilisateur qui les a rédigées. ' Consulter_critique ' fait appel à 'CritiqueDao.find_by_livre et à 'LikeDao.count_by_critique' pour renvoyer les critiques accompagnées du nombre de likes qui leur sont attribués. Enfin, les méthode 'like' et 'dislike' vérifient si il existe déjà un like ou un dislike avec 'LikeDao.exist' pour éviter les doublons (liker plusieurs fois ou liker et disliker).

4.3 Diagrammes de séquence

Les diagrammes de séquence suivant permettent de détailler la chronologie des échanges entre les différents composants de notre architecture (Utilisateur, Front End, Back End, DAO, API externe) pour trois fonctionnalités de l'application : la suppression d'une fiche de lecture, la sélection d'un livre et la demande de recommandations personnalisées.

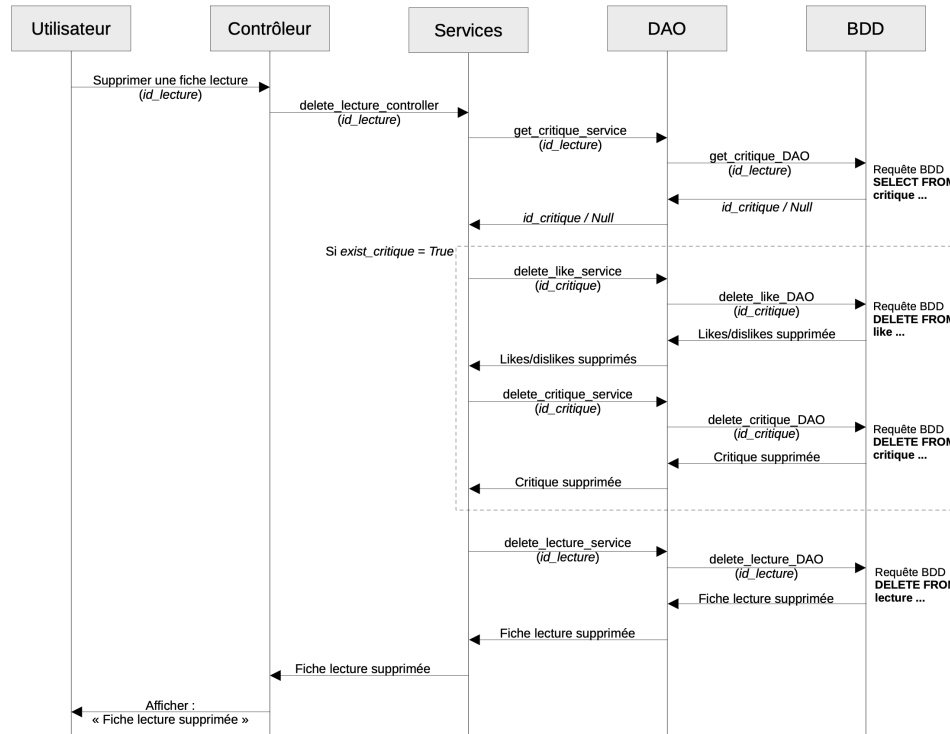


Figure 11 – Diagramme de séquence : supprimer une fiche de lecture

Le but de cette séquence est de supprimer une fiche lecture pour un utilisateur.

Celui-ci a créé auparavant une fiche lecture : il a enregistré un livre avec des informations (statut du livre, note, etc., il a éventuellement ajouté une critique associée). Il souhaite aujourd'hui la supprimer.

Il se situe au départ sur la fiche lecture considérée, et clique sur un bouton « Supprimer la fiche lecture ».

L'information, avec l'`id_lecture` (l'identifiant de la fiche lecture) est réceptionnée par le **Contrôleur**, qui la transmet au **Service**, qui envoie des requêtes au **DAO**.

Dans un premier temps, le **Service** va rechercher dans la base de données l'existence éventuelle d'une **critique** associée à cette lecture. Si une **critique** est trouvée, alors celle-ci sera supprimée, ainsi que d'éventuels likes/dislikes associés.

Ensuite, la fiche lecture est supprimée de la table de données « **lecture** ».

À la fin de toutes ces opérations, le **Service** transmet au **Contrôleur** l'information que la fiche a bien été supprimée, celui-ci affiche à l'écran de l'utilisateur « La fiche lecture a bien été supprimée ».

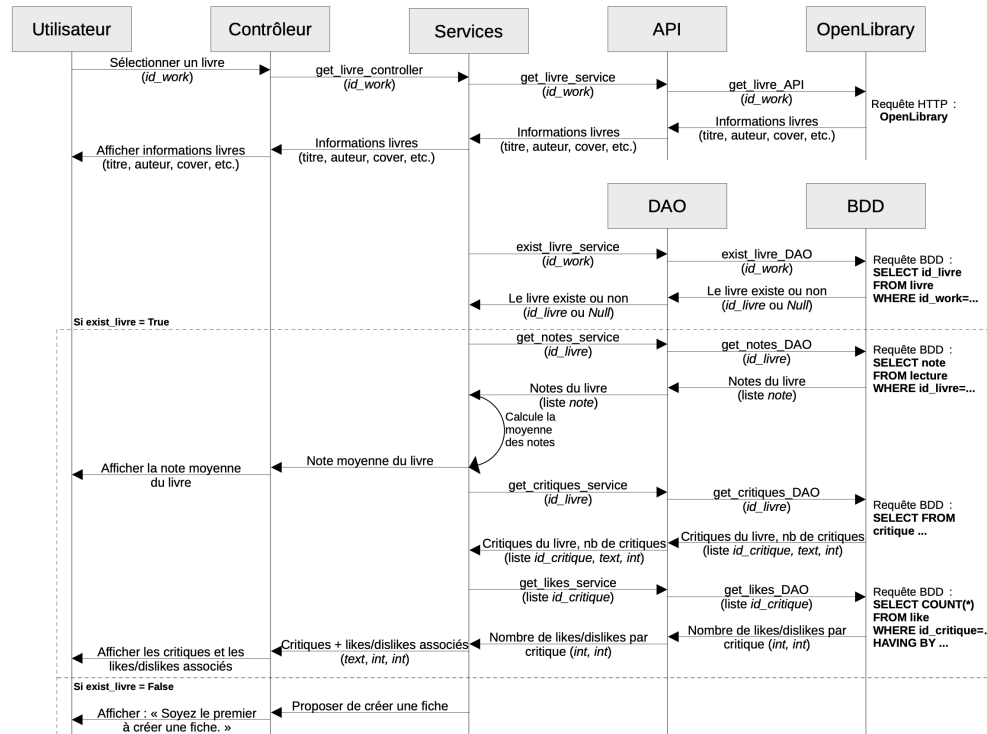


Figure 11 – Diagramme de séquence : sélectionner un livre

Le but de cette séquence est d'afficher toutes les informations disponibles sur un livre sélectionné par un utilisateur.

Celui-ci a par exemple effectué une recherche, a obtenu une liste de livres (correspondant à une liste de **id_work**, les identifiants stockés par **OpenLibrary**), il choisit l'un d'entre eux pour en afficher toutes les informations.

La première étape est de récupérer les informations fournies par **OpenLibrary** sur ce livre : le **Contrôleur** va transmettre au **Service** l'**id_work** considéré, pour utiliser l'**API** qui va effectuer une requête au niveau d'**OpenLibrary**.

De cette requête, on va récupérer les informations pertinentes contenues dans **OpenLibrary** (titre, auteur, date, cover, etc. – la liste précise reste à définir).

Ensuite, il s'agit de fournir les informations propres à notre application Ex-Libris.

Pour cela, le **Service** va transmettre l'`id_work` au **DAO**, qui va rechercher dans la base de données si un livre a déjà été enregistré associé à celui-ci. Si oui, le **DAO** remontera au **Service** un `id_livre` correspondant.

Sinon, il lui proposera d'être le premier à créer une fiche lecture.

À partir de cet `id_livre`, le **Service** va demander au **DAO** de récupérer toutes les notes mises par les utilisateurs à ce livre, pour ensuite en faire la moyenne.

Il va ensuite rechercher les critiques (**critique**) associées à ce livre, leur nombre, et les likes/dislikes (**like**) associés. Si de nombreuses critiques ont été rédigées sur ce livre, il sera possible de n'en choisir que quelques-unes (par exemple, celles avec le plus grand nombre de likes), même si cela n'apparaît pas dans le diagramme pour éviter une surcharge visuelle.

Le **Service** transmet donc au **Contrôleur** les informations suivantes : celles issues d'**OpenLibrary** d'abord ; et si le livre existe déjà dans Ex-Libris, il ajoutera la note moyenne, le nombre de critiques, quelques critiques et les likes/dislikes associés à ces critiques.

Enfin, le **Contrôleur** mettra en forme toutes ces informations pour les afficher sur la page utilisateur.

À noter que dans cette séquence, si le livre n'existe pas dans la base de données Ex-Libris, il n'est pas ajouté à cette base. En effet, il a été convenu par l'équipe projet qu'un ajout ne se fait qu'au moment où un utilisateur enregistre le livre dans sa bibliothèque.

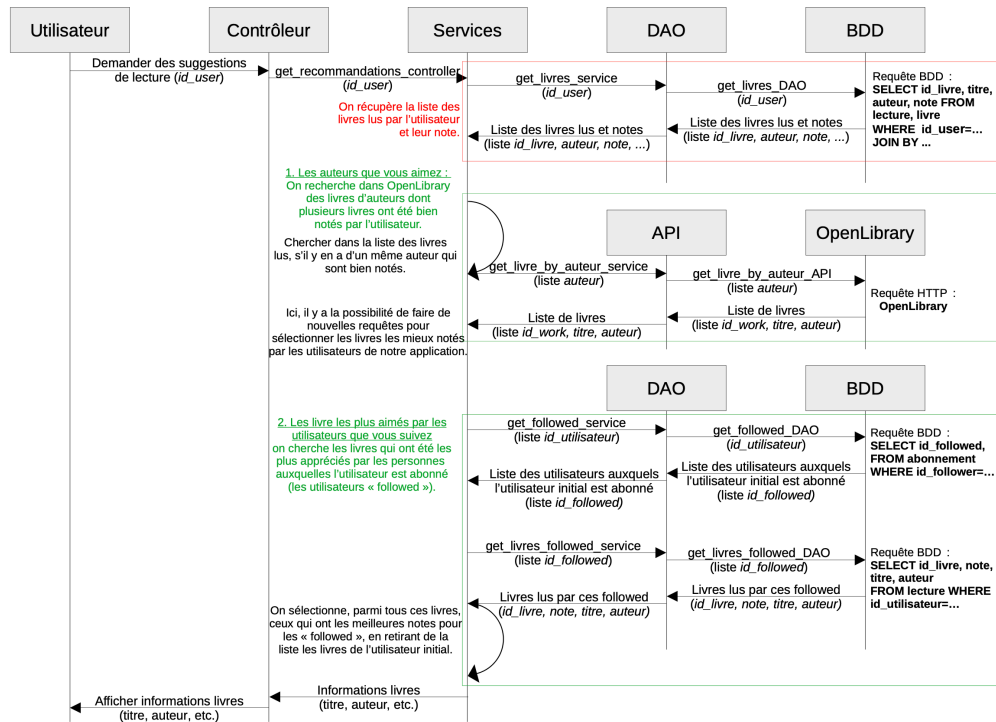


Figure 11 – Diagramme de séquence : sélectionner un livre

Le but de cette séquence est de recommander à l'utilisateur des livres qu'il n'a pas enregistrés et qui seraient susceptibles de lui plaire.

Il se situe sur sa page d'accueil, et clique sur un bouton « Recommandations personnelles ».

La première étape est de récupérer la liste des livres lus par cet utilisateur (`id_livre`), ainsi que les notes associées à chacun de ces livres (`note`). On se sert pour cela de la table `lecture`, qui enregistre les lectures d'un utilisateur.

Nous envisageons ensuite plusieurs possibilités pour effectuer des recommandations.

Premier type de recommandations : "Les auteurs que vous aimez"

Si dans les livres lus par l'utilisateur, plusieurs livres d'un même auteur sont bien notés, d'autres livres du même auteur lui seront proposés.

Le **Service** va donc essayer d'identifier dans la liste de livres de l'utilisateur les auteurs les plus appréciés. S'il trouve des candidats, il va interroger **OpenLibrary**, via l'**API**, pour rechercher d'autres livres des mêmes auteurs.

Il est également possible de rechercher, parmi ces livres d’auteurs appréciés par l’utilisateur, ceux qui ont les meilleures notes dans notre application Ex-Libris, mais cela n’apparaît que comme commentaire sur le diagramme de séquence pour ne pas alourdir le schéma.

Deuxième type de recommandations : "Les livres les plus aimés par les utilisateurs que vous suivez"

Cette fois-ci, nous allons utiliser la fonctionnalité d’abonnement pour offrir des recommandations à l’utilisateur.

Dans un premier temps, il s’agira de récupérer la liste des utilisateurs auxquels celui-ci est abonné (`id_followed` dans la table **Abonnement**).

Ensuite, l’application cherchera, dans les lectures de ces “followed”, les livres les mieux notés, et les proposera à l’utilisateur.

À la fin de l’exécution des algorithmes de recommandations au niveau du **Service**, celui-ci va transmettre au **Contrôleur** une liste des livres recommandés pour l’utilisateur (**titre**, **auteur**, etc.), que ce dernier va mettre en forme et afficher pour l’utilisateur.

Autres types de recommandations (en projet) :

Nous venons de présenter 2 méthodes de recommandation, à titre d’illustration, mais nous avons le projet d’en développer d’autres.

En effet, les recommandations précédentes nécessitent des conditions (avoir noté plusieurs livres d’un même auteur pour la première, avoir des abonnements pour la seconde). Nous envisageons de faire des recommandations du type : “les livres les mieux notés” ou “les livres suscitant le plus de critiques”.

Nous avons conscience que “les livres les mieux notés” peut susciter des requêtes assez lourdes, nous avons donc envisagé la possibilité d’ajouter un champ `note_moyenne` à la table **livre**, qui pourrait être mis à jour à chaque nouvelle note sur un livre.

Un autre type d’algorithme envisagé serait de recouper les livres lus et bien notés entre l’utilisateur initial et d’autres utilisateurs de l’application. Nous obtiendrions alors des utilisateurs avec des goûts similaires à l’utilisateur initial.

Ainsi, en prenant les livres bien notés de ces utilisateurs avec des goûts similaires, nous pourrions alors faire de nouvelles suggestions à l’utilisateur.

Encore une fois, nous avons conscience de la charge potentielle de telles requêtes, et réfléchissons aux possibilités pour les alléger.

partie 1 : Cahier des Charges partie 2 : Maquette de l'application

Suivi projet Ex-Libris

Fait

- Arno : diagramme de séquence (partie 2)
- Paul : diagramme de Gantt (partie 1)
- Paul : présentation de l'API (partie 1)
- Paul : répartition des tâches (partie 1)
- Cecile : Diagramme de cas d'utilisation (partie 1)

En cours

- Moussa : diagramme BDD (partie 2)
- Axel : diagramme d'activité (partie 1)
- Anne-Camille : diagramme de classe business (détail + explication des classes) (partie 2)
- Paul : Introduction

À faire

- Partie répartition des tâches et présentation de l'équipe (partie 2)
- Diagramme de package (DAO / Services / Business / View) (partie 2)
- Détail d'un service (partie 2)
- Présentation de l'API (partie 1) ...